

Lecture 19: Introduction to MIPS Assembly Language & MIPS Architecture

Learning Objectives

By the end of this lecture, students will be able to:

- Understand what Assembly Language is.
 - Explain why Assembly Language is needed.
 - Differentiate between High-Level Language, Assembly Language, and Machine Language.
 - Understand what MIPS is.
 - Explain the MIPS architecture.
 - Identify the major components of a MIPS processor.
 - Understand the instruction execution cycle in MIPS.
 - Become familiar with the MIPS programming environment (MARS/SPIM).
-

1. Introduction

Teaching Script

First 30-Second SEO Phrase

Naturally mention:

"In today's Computer Organization and Assembly Language lecture, we'll begin the MIPS Assembly Language module by understanding what MIPS is, why Assembly Language is important, the MIPS architecture, and how the processor executes instructions step by step."

So far, we've learned how computers represent numbers, perform binary arithmetic, and how the CPU processes data internally.

Now it's time to answer one of the most interesting questions:

How do we actually communicate with the processor?

When we write programs in C++, Python, or Java, the processor cannot understand them directly.

The CPU only understands machine instructions made up of binary numbers.

Writing directly in binary is extremely difficult, so programmers use **Assembly Language**, which provides readable names for machine instructions.

In this module, we'll learn **MIPS Assembly Language**, one of the most popular architectures used for teaching Computer Organization around the world.

By the end of this module, you'll be able to write complete MIPS programs and understand exactly how the CPU executes every instruction.

Let's begin.

2. What is Assembly Language?

Assembly Language is a **low-level programming language** that provides a human-readable representation of machine instructions.

Instead of writing:

```
00100000000010000000000000000101
```

You write:

```
addi $t0, $zero, 5
```

Both represent the same instruction, but Assembly Language is much easier for humans to understand.

3. Why Do We Need Assembly Language?

Imagine writing every instruction in Binary.

Example:

10101100100100110000111111111111

It would be:

- Difficult to read
- Difficult to debug
- Easy to make mistakes

Assembly Language replaces binary instructions with **mnemonics**, making programming much easier.

Examples:

add

sub

lw

sw

beq

j

These short names are called **mnemonics**.

4. Levels of Programming Languages

Machine Language (Lowest Level)

Example:

101010101001001010100101010

Characteristics:

- Binary only
 - Executed directly by the CPU
 - Difficult for humans
-

Assembly Language

Example:

add \$t0, \$t1, \$t2

Characteristics:

- Uses mnemonics
 - Easy to read
 - Requires an **Assembler** to convert into machine code
-

High-Level Languages

Examples:

- C
- C++
- Java
- Python

Example:

int sum = a + b;

Characteristics:

- Easy for humans
 - Hardware independent
 - Requires a compiler or interpreter
-

5. Translation Process

Explain the flow using this diagram:

High-Level Language

|



Compiler

|



Assembly Language

|



Assembler

|



Machine Language

|



CPU

Important Note: In practice, many compilers generate machine code directly or generate assembly as an intermediate step, but conceptually this diagram helps students understand the translation process.

6. What is MIPS?

MIPS stands for:

Microprocessor without Interlocked Pipeline Stages

MIPS is a **RISC (Reduced Instruction Set Computer)** architecture designed to execute a small, simple set of instructions efficiently.

It is widely used for:

- Learning Computer Organization
 - Learning Assembly Language
 - Understanding processor architecture
 - Embedded systems research
-

7. Why Do Universities Teach MIPS?

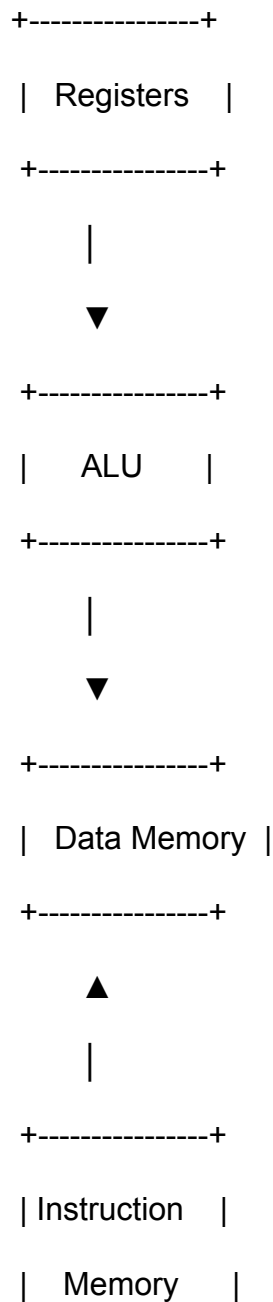
MIPS is chosen because:

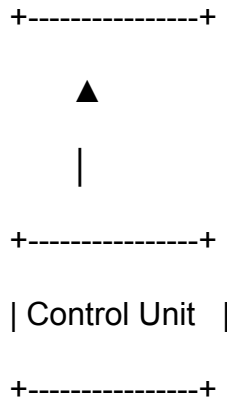
- Simple instruction set
- Easy syntax
- Easy to understand
- Consistent instruction format
- Excellent for learning processor concepts

Even though modern computers may use x86-64 or ARM, the concepts learned in MIPS apply to many architectures.

8. MIPS Architecture Overview

A basic MIPS processor consists of:





9. Main Components

1. Registers

- Small, fast storage locations inside the CPU.
- Hold operands and intermediate results.
- MIPS has **32 general-purpose registers**.

Examples:

\$t0

\$t1

\$s0

\$a0

\$v0

\$sp

\$ra

Explain that the next lecture will discuss these registers in detail.

2. ALU (Arithmetic Logic Unit)

Responsible for:

- Addition
 - Subtraction
 - AND
 - OR
 - Comparisons
 - Shift operations
-

3. Instruction Memory

Stores the program instructions.

Example:

add

lw

sw

beq

4. Data Memory

Stores program data such as variables and arrays.

5. Control Unit

The Control Unit:

- Fetches instructions.
 - Decodes instructions.
 - Generates control signals.
 - Coordinates all CPU components.
-

10. MIPS Instruction Execution Cycle

Every MIPS instruction follows the same basic cycle:

Fetch



Decode



Execute



Memory Access (if needed)



Write Back

Step 1 – Fetch

The CPU retrieves the next instruction from Instruction Memory.

Step 2 – Decode

The Control Unit identifies:

- Instruction type
- Registers
- Immediate values

Step 3 – Execute

The ALU performs the required operation.

Step 4 – Memory Access

If the instruction is **lw** or **sw**, Data Memory is accessed.

Step 5 – Write Back

The result is written into a destination register.

11. Introduction to MARS Simulator

Before writing MIPS programs, students need a simulator.

Recommend **MARS (MIPS Assembler and Runtime Simulator)** because:

- Free
- Easy to use
- Beginner-friendly
- Supports debugging
- Visual register display

Mention that the installation and interface will be covered in the next lecture.

12. Real-Life Analogy

Imagine a restaurant:

- **Instruction Memory** = Recipe book
- **Control Unit** = Chef reading instructions
- **Registers** = Kitchen bowls holding ingredients
- **ALU** = Chef preparing the food
- **Data Memory** = Refrigerator storing ingredients

The chef follows one recipe step at a time, just as the CPU executes one instruction at a time.

13. Common Mistakes

Students often think:

✗ Assembly Language is Machine Language.

✓ Assembly Language is translated into Machine Language by an assembler.

✗ MIPS is a programming language.

✓ MIPS is a processor architecture that uses MIPS Assembly Language.

✗ Registers and RAM are the same.

✓ Registers are much smaller and significantly faster than RAM.



Lecture Summary

Today, we began the MIPS Assembly Language module by learning what Assembly Language is, why it is needed, and how it differs from Machine Language and High-Level Languages.

We introduced the MIPS architecture, explored its major components, and understood how a processor executes instructions through the fetch–decode–execute cycle. We also discussed the role of the assembler and why MIPS is widely used in Computer Organization courses.

In the next lecture, we'll dive deeper into **MIPS Registers**, learning about the 32 general-purpose registers, their naming conventions, and how each register is used in Assembly Language programming.

Thank you for watching, and I'll see you in the next lecture.
